

RDI on Redis Cloud quick start

Learn how to create a data pipeline between a PostgreSQL source database created with Terraform and a Redis Cloud target database.

Redis Cloud

This is a snapshot of the Redis Docs from July 13, 2026.

See <https://redis.io/docs/latest/operate/rc/rdi/quick-start/> for the current version.

The [rdi-cloud-automation GitHub repository](#) contains a Terraform script that quickly sets up a PostgreSQL source database on an EC2 instance and all required permissions and network setup to connect it to a Redis Cloud target database.



Note:

This guide is for demonstration purposes only. It is not recommended for production use.

Prerequisites

To follow this guide, you need to:

1. Create a [Redis Cloud Pro database](#) hosted on Amazon Web Services (AWS). Turn on Multi-AZ replication and [manually select the availability zones](#) when creating the database.
2. Install the [AWS CLI](#) and set up [credentials for the CLI](#).
3. Install [Terraform](#).

Create a data integration workspace

Before you can create your first Data Integration pipeline for a Redis Cloud subscription, you must first deploy the cloud infrastructure needed to host the pipeline and run the workers associated with the pipeline. In Redis Cloud, this is called a **Workspace**. See [Create and manage Data Integration workspace](#) for more information.

To create a Data Integration workspace for an existing [Pro subscription](#):

1. From the Redis Cloud console, select **Data Integration** from the left-hand menu. If you don't have any workspaces yet, select **Create workspace** to go to the **Create workspace** page.

A blue rectangular button with rounded corners and the text "Create workspace" in white.

If you already have a workspace deployed, you'll see your current workspaces. Select **New workspace** to go to the **Create workspace** page.

A blue rectangular button with rounded corners, a white plus icon, and the text "New workspace" in white.

You can also go to the **Data Integration** tab from your subscription or database page and select **Create workspace** to go to the **Create workspace** page for your subscription.

A blue rectangular button with rounded corners and the text "Create workspace" in white.

2. Select your Pro subscription from the list if it's not already selected.

Create workspace

A workspace runs your pipelines in a selected Pro subscription. This is a one-time setup per subscription.

Select Pro subscription

subscription-name



3. A **Data Integration subnet (CIDR)** is automatically generated for you. If, for any reason, a CIDR is not generated, enter a valid CIDR that does not conflict with your applications or other databases.

The Data Integration workspace uses a dedicated subnet to avoid IP conflicts, including in environments using VPC peering.

Data integration subnet (CIDR)

192.168.8.0/22



i Provisioning runs in the background and may take some time. You can continue setting up the pipeline while it runs.

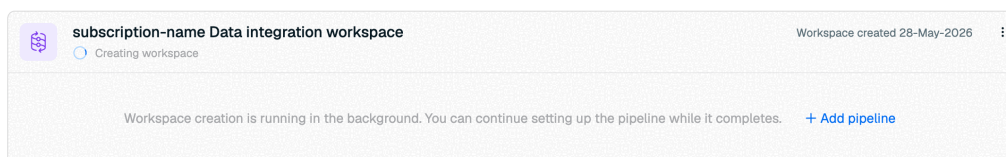
4. Select **Create workspace** to create your workspace.

Create workspace

Your workspace will be created in the background. You can select **Create pipeline** to create your pipeline while the workspace is provisioning, or you can select **Create pipeline later** to go back to the Redis Cloud console.

Get required ARNs

1. On the Redis Cloud console, go to your target database and select the **Data Integration** tab.
2. Select **Add pipeline**.



3. Select **PostgreSQL** as the source database type.

Source

Select your source database type and provide a name.

☐ ☐ ☐ ☐ ☒

ORACLE

MariaDB

MySQL

SQL Server

PostgreSQL

Source name ⓘ

- Enter a name for your source database in the **Source name** field. This is a name for the source database that will appear on Redis Cloud.
- Select **Continue to source** to move to the Source configuration step.

Continue to source

- Under **Source connectivity**, save the provided ARN. This will be the `redis_privatelink_arn` you will need later.

Create a Private Link in the AWS console.
Grant access to the Redis role ARN below, then provide the service name.

[Use automation example ↗](#)
[Read more ↗](#)

Role ARN

arn:aws:iam::123456789012:role/redis-data-pipeline ⓘ

Use this ARN in the allowed list when creating the Private Link in AWS.

Availability zones

us-east-1a us-east-1b us-east-1c ⓘ

Use one of these zones when creating the private link.

- Under **Secrets**, save the provided ARN. This will be the `redis_secrets_arn` you will need later.

Secrets Not configured ⓘ

Provide source secret using AWS secret manager. Enter the secret ARN once created using the ARNs below.

Role ARN

arn:aws:iam::659608289625:role/redis-data-pipeline-secrets-role ⓘ

Grants Redis Cloud access to AWS Secrets Manager.

Create the source database and network resources

- Clone or download the [rdi-cloud-automation GitHub repository](#).
- In a terminal window, go to the `examples/aws-ec2-privatelink` directory.
- Run `terraform init` to initialize the Terraform working directory.
- Open the `example.tfvars` file and edit the following variables:

4. Open the `example.tfvars` file and edit the following variables.

- `region`: The AWS region where your Redis Cloud database is deployed.
- `azs`: The availability zone IDs where your Redis Cloud database is deployed.
- `port`: The port number for the new PostgreSQL source database.
- `name`: A prefix for all of the created AWS resources.
- `redis_secrets_arn`: The source database credentials and certificates ARN from the Redis Cloud console.
- `redis_privatelink_arn`: The PrivateLink ARN from the Redis Cloud console.

5. To view the configuration, run:

```
terraform plan -var-file=example.tfvars
```

6. To create the AWS resources, run:

```
terraform apply -var-file=example.tfvars
```

This example creates the following resources on your AWS account:

- An AWS KMS key with the required permissions for RDI
- A VPC with a public and private subnet and all necessary route tables
- An EC2 instance running a PostgreSQL database with a security group that allows access from Redis Cloud
- An AWS Secrets Manager secret for the PostgreSQL database credentials
- A Network Load Balancer (NLB), a listener, and target group to route traffic to the EC2 instance with AWS PrivateLink
- An AWS PrivateLink endpoint service for the PostgreSQL database

Creating the AWS resources will take some time. After the resources are created, you'll be able to view them in the AWS management console.

Save the following outputs:

- `database`: The name of the PostgreSQL database.
- `port`: The port number for the PostgreSQL database.
- `secret_arn`: The ARN of the AWS Secrets Manager secret for the PostgreSQL database credentials.
- `vpc_endpoint_service_name`: The name of the AWS PrivateLink endpoint service for

the PostgreSQL database.

If you lose any outputs, run `terraform output` to view them again.

Resume pipeline setup

1. Return to the [Redis Cloud console](#). Go to your target database and select the **Data Integration** tab.
2. You'll see a draft pipeline in the workspace you created. Select **More actions > Resume pipeline setup** to continue with pipeline setup.



3. Continue to the **Source configuration** step.
4. In the **Source connectivity** section, enter the `vpc_endpoint_service_name` output in the **PrivateLink** service name field.

Source connectivity Not configured

Configure how Redis Cloud connects to your source database.

☒ **AWS Private Link**
Private AWS connectivity

☐ **Public Endpoint**
Public internet access

Create a Private Link in the AWS console.
Grant access to the Redis role ARN below, then provide the service name. [Use automation example ↗](#) [Read more ↗](#)

Role ARN
arn:aws:iam::123456789123:role/redis-data-pipeline

Availability zones
us-east-1a us-east-1b us-east-1c

Use this ARN in the allowed list when creating the Private Link in AWS. Use one of these zones when creating the private link.

Private Link service name

Enter the service name created in AWS.

5. Select **Connect to Private Link** to test your Private Link connectivity. This will take a few minutes, but you can continue while it's testing.
6. In the **Secrets** section, enter the `secret_arn` output in the **Credentials secret ARN** field.

Secrets Not configured

Provide source secret using AWS secret manager. Enter the secret ARN once created using the ARNs below.

Role ARN
`arn:aws:iam::659608289625:role/redis-data-pipeline-secrets-role` ⓘ
Grants Redis Cloud access to AWS Secrets Manager.

Authentication (required)
Provide credentials to access your source database.

Credentials Secret ARN
`arn:aws:secretsmanager:us-east-1:123456789012:secret:redis-rdl-abc234!` [How to create this secret](#) ⓘ
Database credentials stored in AWS Secrets Manager.

Transit security (Optional)
Add encryption to secure data in transit.

☒ **Unencrypted**
Unsecured transit

☐ **TLS**
Recommended for most environments.

☐ **mTLS**
Recommended for regulated environments.

[Validate](#)

7. Select **Validate** to check that Redis Cloud can access your secrets.
8. In the **Source configuration** section, enter the terraform outputs in the following fields.
 - **Database:** `database`
 - **Port:** `port`
9. Select **Test source** to test Redis Cloud's connection with the source database. After the test completes, select **Continue to dataset**.

[Continue to dataset](#)

10. In the **Schemas** section, select the schema(s) you want to migrate to the target database from the list.

Schemas (1)

☒

Q Search schemas...

☒

public

11/11 tables

>

All 1 schemas selected

Tables (12) | Schema: public

☒

Q Search tables...

☒

album

3/3 columns

:

☒

artist

2/2 columns

:

☒

customer

13/13 columns

:

☒

employee

15/15 columns

:

☐

employees

6 columns

:

☒

genre

2/2 columns

:

☒

invoice

9/9 columns

:

☒

invoiceline

5/5 columns

:

☒

mediatype

2/2 columns

:

11 tables selected

11. When you select a schema, you will see its tables in the **Tables** section. Redis Cloud will automatically select all tables for import. You can de-select any columns you do not wish to import to your Redis database.
12. Select a table to view its columns in the **Columns** section. You can de-select any columns you do not wish to import.

Tables (12) | Schema: public

☒

Q Search tables...

☒

album

2/3 columns

:

☒

artist

2/2 columns

:

☒

customer

13/13 columns

:

☒

employee

15/15 columns

:

☐

employees

6 columns

:

☒

genre

2/2 columns

:

Columns (3) | Table: album

☒

Q Search columns...

☒

albumid

PK

INT4

☐

artistid

INT4

☒

title

VARCHAR

☒ yente	4/4 columns	:
☒ invoice	9/9 columns	:
☒ invoiceline	5/5 columns	:
☒ mediatype	2/2 columns	:
11 tables selected		
		2 columns selected

13. Select Continue to transformations to move to the Transformations step.

Continue to transformations

14. Select how your records will be stored in Redis. You can choose Hash or JSON.

Transformations

Default data structure ⓘ

How records are stored in Redis.

☐ Hash ☒ JSON

Transformation jobs (optional)

Add transformation jobs for tables that require custom transformations.

Upload jobs

Processor properties

Fine-tune how incoming data is processed and written to Redis.

[Edit advanced properties](#)

No advanced properties configured.

15. Review the tables you selected in the Review and deploy step. If everything looks correct, select Deploy pipeline to start ingesting data from your source database.

Deploy pipeline

At this point, the data pipeline will ingest data from the source database to your target Redis database. This process will take time, especially if you have a lot of records in your source database.

After this initial sync is complete, the data pipeline enters the *change streaming* phase, where changes are captured as they happen. Changes in the source database are added to the target within a few seconds of capture.

You can view the status of your data pipeline in the **Data pipeline** tab of your database. See [View and edit data pipeline](#) to learn more.

Delete sample resources



Warning:
Make sure to [delete your data pipeline](#) before deleting the sample resources.

To delete the sample resources created by Terraform, run:

```
terraform destroy -var-file=example.tfvars
```



RATE THIS PAGE



[Back to top](#) ↑